

Unit tests de las funcionalidades de la aplicación Gymly



Walter Julián Fernández González

Jarol David Zuluaga Grau

Sebastian Olaya Sanchez

Alessandro Garzon Melo

Universidad Nacional de Colombia

Facultad de Ingeniería

Prof. Oscar Eduardo Alvarez Rodriguez

Ingeniería de Software I

2026

Unit tests

1. Autenticación (Walter Fernández).

a. Autenticación de usuario inactivo

Caso límite: Intento de inicio de sesión de un usuario que existe en el sistema, pero su cuenta fue desactivada (active = false). Debe denegar el acceso y lanzar una excepción.

Test

```
@Test @wxtierj
fun `login should throw InvalidCredentialsException when user is not active`() {
    val request = LoginRequest(identifier = "test@user.com", password = "password123")
    val inactiveUser = User(
        id = 100, roleId = 1, name = "Test", lastname = "User",
        document = "12345678", phone = "555-1234", email = "test@user.com",
        passwordHash = "hashedPassword", active = false
    )

    `when` ( methodCall = userRepository.findByEmail("test@user.com")).thenReturn( value = inactiveUser)

    val exception = assertThrows<InvalidCredentialsException> {
        authService.login(request)
    }

    assertEquals( expected = "Invalid credentials", actual = exception.message)
    verify( mock = passwordEncoder, mode = never()).matches( rawPassword = anyString(), encodedPassword = anyString())
}
```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 AuthService	100% (1/1)	30% (3/10)	17% (11/62)	14% (4/28)

b. Renovación de sesión

Caso límite: El cliente envía un Refresh Token cuyo tiempo de vida (`expiresAt`) ya ha pasado. La lógica debe capturar esto y abortar el refresh, forzando un re-login.

Test

```
@Test @wxtierj
fun `refreshAccessToken should throw InvalidCredentialsException when token is expired`() {
    val expiredTokenStr = "expired-refresh-token"
    val request = RefreshTokenRequest(refreshToken = expiredTokenStr)
    val expiredToken = RefreshToken(
        id = 1,
        userId = 100,
        token = expiredTokenStr,
        expiresAt = Instant.now().minus(amountToSubtract = 1, unit = ChronoUnit.DAYS),
        createdAt = Instant.now().minus(amountToSubtract = 8, unit = ChronoUnit.DAYS)
    )

    `when` { methodCall = refreshTokenRepository.findByToken(expiredTokenStr).thenReturn(value = expiredToken)

    val exception = assertThrows<InvalidCredentialsException> {
        authService.refreshAccessToken(request)
    }

    assertEquals(expected = "Refresh token has expired", actual = exception.message)
    verify(mock = userRepository, mode = never()).findById(anyInt())
}
```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 AuthService	100% (1/1)	20% (2/10)	14% (9/62)	7% (2/28)

c. Rol no existente.

Caso límite: Se intenta registrar un cliente pero el rol CLIENT no existe en la base de datos (no fue seedeado). El servicio debe abortar con 'RoleNotConfiguredException' sin tocar encriptación ni persistencia.

Test

```
@Test @winterj *
fun `registerClient should throw RoleNotConfiguredException when role is not found`() {
    val request = RegisterRequest(
        name = "Bob",
        lastname = "NoRole",
        email = "bob@test.com",
        document = "11223344",
        password = "password123"
    )

    `when` { methodCall = roleRepository.findByName(AppConstants.ROLE_CLIENT)).thenReturn(value = null)

    assertThrows<RoleNotConfiguredException> {
        authService.registerClient(request)
    }

    verify(mock = passwordEncoder, mode = never()).encode(rawPassword = anyString())
    verify(mock = userRepository, mode = never()).save(entity = any<type = User::class.java>())
}
```

Coverage

Element ^	Class, %	Method...	Line, %	Branch,...
 AuthService	100% (1/...	20% (2/10)	11% (7/62)	3% (1/28)

2. Estadísticas de asistencias (Sebastián Olaya).

a. Agrupación por horas

Caso límite: múltiples asistencias en la misma hora deben sumarse en un solo punto, y horas sin asistencias deben retornar 0 sin lanzar excepción.

```

@Test
fun `groups and counts attendances by hour correctly`() {
    val today = LocalDate.now()
    whenever( methodCall = attendanceRepository.findByDateBetween(any(), any())) .thenReturn( value = listOf(
        makeAttendance( dateTime = today.atTime( hour = 9, minute = 0)),
        makeAttendance( dateTime = today.atTime( hour = 9, minute = 30)),
        makeAttendance( dateTime = today.atTime( hour = 9, minute = 59)),
        makeAttendance( dateTime = today.atTime( hour = 14, minute = 0)),
        makeAttendance( dateTime = today.atTime( hour = 14, minute = 45))
    ))

    val response = attendanceStatsService.calculateAttendanceMetrics( period = StatPeriod.HOURS_DAY, offset = 0)

    assertEquals( expected = 18, actual = response.points.size)
    assertEquals( expected = "06:00", actual = response.points.first().label)
    assertEquals( expected = "23:00", actual = response.points.last().label)
    assertEquals( expected = 3L, actual = response.points.first { it.label == "09:00" }.count)
    assertEquals( expected = 2L, actual = response.points.first { it.label == "14:00" }.count)
    assertEquals( expected = 0L, actual = response.points.first { it.label == "10:00" }.count)
    assertEquals( expected = 0L, actual = response.points.first { it.label == "22:00" }.count)
}

```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 AttendanceStatsService	100% (1/1)	50% (3/6)	37% (26/69)	34% (11/32)

b. Análisis Hora pico

Caso límite: Caso límite: cuando no hay asistencias el peakValue debe ser 1 y no 0, para evitar división por cero al escalar las barras en el frontend.

Test

```

@Test
fun `peak value matches the highest attendance count across all points`() {
    val today = LocalDate.now()
    whenever( methodCall = attendanceRepository.findByDateBetween(any(), any()))
        .thenReturn( value = listOf(
            makeAttendance( dateTime = today.atTime( hour = 10, minute = 0)),
            makeAttendance( dateTime = today.atTime( hour = 10, minute = 15)),
            makeAttendance( dateTime = today.atTime( hour = 10, minute = 45)),
            makeAttendance( dateTime = today.atTime( hour = 15, minute = 0))
        ))

    val response = attendanceStatsService.calculateAttendanceMetrics( period = StatPeriod.HOURS_DAY, offset = 0)

    assertEquals( expected = 3L, actual = response.peakValue)
    assertEquals( expected = response.points.maxOf { it.count }, actual = response.peakValue)

    whenever( methodCall = attendanceRepository.findByDateBetween(any(), any()))
        .thenReturn( value = emptyList())

    val emptyResponse = attendanceStatsService.calculateAttendanceMetrics( period = StatPeriod.HOURS_DAY, offset = 0)

    assertEquals( expected = 1L, actual = emptyResponse.peakValue)
    assertTrue(emptyResponse.points.all { it.count == 0L })
}

```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 AttendanceStatsService	100% (1/1)	50% (3/6)	37% (26/69)	34% (11/32)

c. Evitar fechas nulas

Caso límite: un registro con date=null llega desde la base de datos por un error de inserción. El servicio debe filtrar silenciosamente en lugar de lanzar NullPointerException al intentar acceder a .toLocalDateTime().hour sobre null.

Test

```

@Test
fun `ignores attendances with null date without throwing exception`() {
    whenever( methodCall = attendanceRepository.findByDateBetween(any(), any()))
        .thenReturn( value = listOf(
            Attendance(userId = 1, date = null),
            Attendance(userId = 2, date = null),
            makeAttendance( dateTime = LocalDate.now().atTime( hour = 10, minute = 0), userId = 3)
        ))

    val response = attendanceStatsService.calculateAttendanceMetrics( period = StatPeriod.HOURS_DAY, offset = 0)

    assertEquals( expected = 1L, actual = response.points.first { it.label == "10:00" }.count)
    assertEquals( expected = 0L, actual = response.points.filter { it.label != "10:00" }.sumOf { it.count })
    assertEquals( expected = 1L, actual = response.peakValue)
}

```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 AttendanceStatsService	100% (1/1)	50% (3/6)	37% (26/69)	37% (12/32)

3. Alertas Visuales (Alessandro Garzón)

a. Dado un documento, decide el color del “semáforo”

Caso límite: Este caso límite evalúa el comportamiento del sistema ante una frontera de existencia, donde se introduce un **documento de identidad no registrado** ("9999") que provoca que el repositorio de usuarios devuelva un valor nulo (null).

Test

```
@Test
fun returnsRedWhenUserDoesNotExist() {
    val document = "9999"
    whenever( methodCall = userRepository.findByDocument(document)).thenReturn( value = null)

    val result = service.checkAccessColor(document)

    assertEquals( expected = "RED", actual = result.statusColor)
    assertEquals( expected = document, actual = result.document)
    assertNull(result.userName)
    assertEquals( expected = 0L, actual = result.daysRemaining)
    assertEquals( expected = "Usuario no registrado en el sistema.", actual = result.message)
    verify( mock = membershipRepository, mode = never()).findFirstByIdOrderByEndDateDesc( userId = any())
}
```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 VisualAlertService	100% (1/1)	28% (2/7)	16% (5/31)	7% (1/14)

b. Obtener la última membresía del usuario

Caso límite: un usuario está correctamente registrado pero no posee historial de membresías asociadas (el conjunto de datos está vacío)

Test

```
@Test
fun returnsRedWhenUserHasNoMembership() {
    val user = buildUser()
    whenever( methodCall = userRepository.findByDocument(user.document)).thenReturn( value = user)
    whenever( methodCall = membershipRepository.findFirstByIdOrderByEndDateDesc( userId = user.id!!)).thenReturn( value = null)

    val result = service.checkAccessColor(user.document)

    assertEquals( expected = "RED", actual = result.statusColor)
    assertEquals( expected = "El usuario no cuenta con ninguna membresía.", actual = result.message)
    assertEquals( expected = 0L, actual = result.daysRemaining)
    assertEquals( expected = user.id, actual = result.userId)
}
```

Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 VisualAlertService	100% (1/1)	28% (2/7)	16% (5/31)	7% (1/14)

c. Decidir el estado a partir de status más la fecha fin

Caso límite: Tres límites críticos: primero, la transición del umbral de tiempo, comprobando que si la fecha de fin es exactamente hoy (`LocalDate.now()`), la membresía se mantiene vigente con alerta "YELLOW", pero si fue ayer, cambia de inmediato a "RED" (vencida). Segundo, evalúa la prioridad de las reglas de negocio, garantizando que un estado administrativo no activo (como "FROZEN") actúa como un cortocircuito que invalida y anula el acceso con color "RED", sin importar que la fecha de vencimiento sea en el futuro.

Test

```
@Test
fun endDateTodayIsStillValid() {
    val user = buildUser()
    val membership = buildMembership(endDate = LocalDate.now())
    whenever( methodCall = userRepository.findByDocument(user.document)).thenReturn( value = user)
    whenever( methodCall = membershipRepository.findFirstByUserIdOrderByEndDateDesc( userId = user.id!!)).thenReturn( value = membership)

    val result = service.checkAccessColor(user.document)

    assertEquals( expected = "YELLOW", actual = result.statusColor)
    assertEquals( expected = 0L, actual = result.daysRemaining)
}
```

```
@Test
fun endDateYesterdayIsExpired() {
    val user = buildUser()
    val yesterday = LocalDate.now().minusDays( daysToSubtract = 1)
    val membership = buildMembership(endDate = yesterday)
    whenever( methodCall = userRepository.findByDocument(user.document)).thenReturn( value = user)
    whenever( methodCall = membershipRepository.findFirstByUserIdOrderByEndDateDesc( userId = user.id!!)).thenReturn( value = membership)

    val result = service.checkAccessColor(user.document)

    assertEquals( expected = "RED", actual = result.statusColor)
    assertEquals( expected = 0L, actual = result.daysRemaining)
    assertEquals( expected = "Membresía vencida el $yesterday.", actual = result.message)
}
```

```
@Test
fun nonActiveStatusOverridesFutureDate() {
    val user = buildUser()
    val membership = buildMembership(
        endDate = LocalDate.now().plusDays( daysToAdd = 30),
        status = MembershipStatus.FROZEN
    )
    whenever( methodCall = userRepository.findByDocument(user.document)).thenReturn( value = user)
    whenever( methodCall = membershipRepository.findFirstByUserIdOrderByEndDateDesc( userId = user.id!!)).thenReturn( value = membership)

    val result = service.checkAccessColor(user.document)

    assertEquals( expected = "RED", actual = result.statusColor)
    assertEquals( expected = "Membresía inactiva (Estado: FROZEN).", actual = result.message)
}
```


Coverage

Element ^	Class, %	Method, %	Line, %	Branch, %
 VisualAlertService	100% (1/1)	85% (6/7)	64% (20/31)	50% (7/14)

4. Gestión de Usuarios (Jarol Zuluaga Grau)

a. Validación de documento duplicado al registrar un cliente nuevo.

Caso límite: El Administrador intenta registrar a un nuevo cliente ingresando un número de documento que ya se encuentra registrado y pertenece a otro usuario en el sistema. El sistema debe bloquear la acción inmediatamente para evitar duplicidad de identidades.

Test

```
@Test new *
fun `registerUser should throw DuplicateDocumentException when document already exists`() {
    val request = UserRequest(name = "Nuevo", lastname = "Cliente", document = "101010", email = "nuevo@test.com")
    val existingUser = User(
        id = 1,
        roleId = 1,
        name = "Antiguo",
        lastname = "Cliente",
        document = "101010",
        phone = "555123",
        email = "antiguo@test.com",
        passwordHash = "hash",
        active = true
    )
    `when` { methodCall = userRepository.findByDocument("101010").thenReturn(p0 = existingUser)
    assertThrows<DuplicateDocumentException> {
        userService.registerUser(request)
    }
    verify { mock = userRepository, mode = never().save(entity = any { type = User::class.java })
}
@Test new *
```

Coverage

Element v	Class, %	Method, %	Line, %	Branch, %
 UserService	100% (1/1)	40% (2/5)	41% (5/12)	30% (3/10)

b. Validación de correo electrónico en uso por otra persona

Caso límite: Durante la actualización del perfil de un cliente (ej. "Juan"), el Administrador ingresa por error un correo electrónico que ya le pertenece a otro cliente activo en el sistema (ej. "María"). El sistema debe detectar el choque de identificadores (IDs) y rechazar el cambio para no comprometer el acceso de la otra persona.

Test

```
@Test new *
fun `updateUser should throw EmailAlreadyInUseException when email belongs to another user`() {
    val userIdToUpdate = 2
    val request = UserRequest(name = "Juan", lastname = "Perez", document = "2222", email = "maria@test.com")
    val otherUser = User(
        id = 5,
        roleId = 1,
        name = "Maria",
        lastname = "Gomez",
        document = "5555",
        phone = "555444",
        email = "maria@test.com",
        passwordHash = "hash",
        active = true
    )
    `when` ( methodCall = userRepository.findByEmail("maria@test.com")).thenReturn( p0 = otherUser)
    assertThrows<EmailAlreadyInUseException> {
        userService.updateUser( id = userIdToUpdate, request)
    }
    verify( mock = userRepository, mode = never()).save( entity = any( type = User::class.java))
}
```

Coverage

Element	Class, %	Method, %	Line, %	Branch, %
 UserService	100% (1/1)	40% (2/5)	41% (5/12)	30% (3/10)

c. Soft Delete" y validación de desactivación

Caso Límite: El Administrador intenta dar de baja o "eliminar" a un cliente que **ya había sido desactivado** previamente. Puesto que el sistema no borra registros físicos para proteger el historial de pagos (usa `active = false`), la lógica debe capturar este intento redundante y abortar la operación para ahorrar recursos.

Test

```
@Test new *
fun `deactivateUser should throw UserAlreadyInactiveException when user is already inactive`() {
    val inactiveUserId = 10
    val inactiveUser = User(
        id = inactiveUserId,
        roleId = 1,
        name = "Inactivo",
        lastname = "User",
        document = "9999",
        phone = "555999",
        email = "off@test.com",
        passwordHash = "hash",
        active = false
    )
    `when` { methodCall = userRepository.findById(inactiveUserId) }.thenReturn { p0 = Optional.of( value = inactiveUser) }
    assertThrows<UserAlreadyInactiveException> {
        userService.deactivateUser(inactiveUserId)
    }
    verify( mock = userRepository, mode = never() ).save( entity = any( type = User::class.java) )
}
```

Coverage

Element	Class, %	Method, %	Line, %	Branch, %
 UserService	100% (1/1)	80% (4/5)	91% (11/12)	50% (5/10)